ArmorIQ Policy Isolation

Strategy Brief

Prepared by: ArmorIQ Engineering **Date:** June 2026

The Problem

Our enforcement layer and the AI agent run as the same OS user. The agent can read, write, kill, and modify everything the enforcement can. We need a trust boundary the agent cannot cross.

The Solution: Enforcement Daemon

We move all enforcement into a separate process (`armoriq-policyd`) running as a dedicated system user (`_armoriq`). The agent talks to it over a Unix socket. It cannot modify policy, kill the daemon, or read credentials.

Why a daemon and not in-process enforcement:

- **CBSE prevention.** Configuration-Based Sandbox Escape (CVE-2026-25725, CVSS 7.7) is the #1 attack on AI coding tools. Agents modify `.mcp.json` or `CLAUDE.md` to inject malicious MCP servers. Our daemon is immune – agent config changes don't affect enforcement.
- **Cross-product enforcement.** One daemon serves ArmorClaude, ArmorCopilot, ArmorCodex, and SDK agents. One policy, not three.
- **Persistence.** In-process enforcement dies with the session. The daemon persists, maintains audit logs, and syncs with the cloud backend.
- **Speed.** Cloud-only enforcement adds 50-200ms per decision. The daemon delivers <5ms locally.

- **Trust separation.** The agent cannot be trusted to enforce its own policy. Same process = same trust domain. A separate daemon breaks that.

The OS user separation and MAC profiles are hardening on top. They're not the reason the daemon exists.

The Sudo Problem

If the agent has sudo, it has root. Root bypasses Unix file permissions entirely. It can read policy files, kill the daemon, take ownership of anything.

How we handle it – two tiers:

Tier 1: Base

The daemon runs as `_armoriq`. Policy files are mode 0600, owned by `_armoriq`. The agent communicates via Unix socket. Policy mutations require a `user-session` auth tag that only fires on human terminal input (UserPromptSubmit hook). CSRG Merkle proofs detect any tampering. Cloud sync provides a remote source of truth.

This stops all non-root tampering, CBSE attacks, cross-agent escalation, and config injection.

Tier 2: hardening (one config file)

A single MAC profile – SELinux on RHEL, AppArmor on Ubuntu, or sandbox profile on macOS – dropped by the installer. MAC overrides DAC. Even root is subject to MAC policy.

This additionally stops root-level policy tampering.

That's it. Two tiers.

Capabilities restriction, namespace hiding, immutable attrs, kernel lockdown – these are implementation details inside the MAC profile or systemd unit file. The installer configures them as part of a single hardening step. We don't expose them as separate layers because that's over-engineering and creates tech debt. Each technique is useful, but they compose into one deliverable: the MAC config file.

Why this is enough: - Individual developers don't give agents sudo. Tier 1 is sufficient. - Enterprise teams need "even root can't tamper" for compliance. Tier 2 gives them that. - SELinux has stopped real CVEs: CVE-2019-9213 (mmap escalation blocked in enforcing mode), CVE-2019-13272 (ptrace escalation blocked with deny_ptrace). -

Root can disable SELinux via `setenforce 0`, but kernel lockdown (Linux 5.4+) prevents that. The installer checks and warns.

Why Not Hardware Enclaves

Hardware enclaves (Intel SGX, AMD SEV, ARM TrustZone) protect data from even root and the kernel. We evaluated them and decided against using them as our primary isolation.

Why not: - SGX is deprecated on consumer CPUs (Intel 12th gen+). Our developers won't have it. - macOS has no general-purpose enclave. Apple Secure Enclave only stores keys. - Development complexity is extreme – restricted C, no system calls, no dynamic memory. - Side-channel attacks keep breaking the guarantees (Foreshadow, Plundervolt, SGAxe, CacheOut, AEPIC Leak, Downfall – all with CVEs). - Enclaves can't prevent semantic attacks. "TEE-backed memory isolation does not by itself stop a prompt-injected agent from leaking data through its own outputs or tool calls." (Imperial College, 2026) - No production framework exists for agentic AI policy enforcement. - Overkill for our threat model. We're protecting policy JSON, not nuclear secrets.

Where we can use hardware security: - CSRG signing keys stored in Apple Secure Enclave / TPM 2.0 - Cloud backend in AMD SEV-SNP / Intel TDX VMs - API credentials in OS keychain (hardware-backed)

These are targeted uses for key storage, not policy evaluation.

PAP, IAP, KAP – Our Research Stack

We have three research artifacts. Each operates at a different layer. They are not interchangeable.

PAP (Intent-Preserving Refinement) is a theoretical model. It defines how agent plans should evolve: uncertainty decreases, authority only narrows, never widens. It's the refinement chain `H -> I -> P -> A -> E` (human purpose through to machine effect). PAP is not a daemon or service. It's the theory.

IAP (Intent Assurance Plane) provides execution-level cryptographic verification. Our CSRG Merkle tokens implement IAP concepts – binding policy state to a verifiable chain.

KAP (Kernel Assurance Plane) is a kernel enforcement layer. It takes compiled authority from PAP/IAP and enforces it at the syscall level inside the Linux kernel.

The enforcement daemon (`armoriq-policyd`) implements PAP's refinement invariant and IAP's execution guard as a running service. It is not PAP itself – it's the system that operationalizes these theoretical models.

KAP: With and Without

Without KAP (what we ship now):

```
Agent hooks -> armoriq-policyd (as _armoriq) -> allow/deny
                      |
              OS user isolation + MAC profile (enterprise)
```

This is complete and production-ready.

With KAP (future, when it matures):

```
Agent hooks -> armoriq-policyd (as _armoriq) -> typed predicates
                      |
              PAP-KAP adapter -> kapd -> kapfs -> KAP LSM hooks (kernel)
```

KAP adds kernel-level enforcement below root. Even sudo can't bypass KAP hooks. Authority can only narrow, never widen. Stale epochs fail closed.

Why we wait on KAP: - Requires a kernel overlay. Customers can't brew install it. - Linux only. Doesn't work on macOS. - Prototype runs in a QEMU VM. Not battle-tested. - Positioned for upstream Linux mainline. Waiting is pragmatic. - MAC profiles solve the sudo problem today without kernel modules.

Why we design for KAP now: - Our daemon output maps directly to KAP input. If we build the right interfaces, KAP integration is an adapter, not a rewrite. - Policy internally represented as typed predicates (exec, file, net, IPC) even though current enforcement returns allow/deny. - CSRG tokens are epoch-aware. Our Merkle chain maps to IAP lineage roots. - Nothing is thrown away. KAP is additive.

KAP is a 3-file kernel overlay (~1,771 lines) wired into Linux's LSM framework. This is how SELinux and AppArmor were added. An RFC patch series is ready for LKML submission. For comparison, SELinux is ~100,000 lines. KAP is small and auditable.

Competitive Position

Capability	OPA	Cerbos	Invariant	Guardian Shell	ArmorIQ
Stops root/sudo	No	No	No	Partial	**Yes (MAC)**
CBSE protection	No	No	No	Partial	**Yes**
Authority lineage	No	No	No	No	**Future (KAP)**
Latency	Sub-ms	Sub-ms	Near-zero	Low	**<5ms**
Human-only mutation	N/A	N/A	N/A	Partial	**Yes**
Crypto tamper detection	Bundle signing	No	No	No	**CSRG Merkle**
Cross-platform	Yes	Yes	Yes	Linux only	**macOS + Linux**

Guardian Shell claims to solve TOCTOU races and symlink attacks via Landlock + seccomp + eBPF. These claims were refuted 0-3 in adversarial verification.

Attack Surface: Before and After

Attack	Today	With daemon + MAC	With KAP
Write to policy files	Bypassable regex	**Blocked**	**Blocked**
Kill the daemon	Same user, works	**Blocked**	**Blocked**
Read credentials	Same user, works	**Blocked**	**Blocked**
Send fake mutations via socket	Can send anything	**Blocked** (auth required)	**Blocked**
CBSE (modify agent config)	No guard	**Blocked**	**Blocked**
Inject malicious MCP server	Possible	**Blocked**	**Blocked**

Attack	Today	With daemon + MAC	With KAP
Cross-agent escalation	Possible	**Blocked**	**Blocked**
Sudo escalation	Password required	**Blocked** (MAC)	**Blocked** (kernel)
Authority drift	No detection	No detection	**Blocked** (KAP unique)

Today: 4/16 attacks blocked. With daemon + MAC: 14/16. With KAP: 15/16.

Implementation Plan

Phase	What	Delivers
0 (now)	Ship current v0.3	Working policy system, 195 tests
1	Extract daemon to armoriq-policyd repo	Standalone daemon with socket API
2	System service installer	OS-level isolation, `_armoriq` user, launchd/systemd
3	ArmorClaude thin client	ArmorClaude shrinks to ~200 lines
4	MAC profiles + socket auth hardening	SELinux/AppArmor/macOS sandbox, peer creds
5	Cross-product client library	npm + Python packages
6 (future)	KAP integration	PAP-KAP + IAP-KAP adapters

Risks

Risk	Impact	Mitigation
sudo install friction	High	Homebrew formula; observe-only mode without daemon
macOS Gatekeeper blocks unsigned daemon	Medium	Apple Developer ID signing

Risk	Impact	Mitigation
MAC misconfiguration	High	Installer auto-detects and configures
CBSE on ArmorIQ itself	High	Daemon owns all config; agent config irrelevant
Competitor ships first	Medium	Ship v0.3 now; daemon is v1.0
KAP never goes upstream	Medium	Without-KAP stack is complete and sufficient